

7/23/23, 2:31 AM

Codio - Monitoring and Analyzing ChatGPT API Usage and Performance

CodioProjectFileEditFindViewToolsEducationHelpNode versionProject Index (static)Configure...

SANDIPAN-DEY

exerc.pyxTerminal

```
1 import os
2 import openai
3 import secret
4 import time
5 openai.api_key='sk-QVkkVGvk7ose2G3HCQM7T3B1bkFJI8YuDKIMdS8Rk6rTsgIY'
6 # WRITE YOUR CODE HERE
7
8 def Generate(prompts):
9     responses = []
10    for prompt in prompts:
11        # Start the timer
12        start_time = time.time()
13
14        # Make the API call to generate the completion
15        response=openai.Completion.create(
16            engine="text-davinci-002", # Select the engine
17            prompt=prompt,
18            max_tokens=100 # Limit the response length
19        )
20
21        # Calculate the elapsed time
22        elapsed_time = time.time() - start_time
23        responses.append((response.choices[0].text.strip(), elapsed_time))
24    return responses
25
26 prompts = ["Who won the world series in 2020?", "What's the capital of France?", "Who wrote Pride a
27 responses = Generate(prompts)
28
29 for i, (response, time_taken) in enumerate(responses):
30     print(f"Prompt: {prompts[i]}\nResponse: {response}\nTime Taken: {time_taken} seconds\n")
31
```

Guide x

CollapseIntro to GPT-3

Coding Exercise

In this exercise, we'll be writing a function `Generate` which uses OpenAI's API to generate text. This function will take a list of prompts from the user, process each prompt with the OpenAI model, and return the response generated by each prompt, as well as the time it took to generate each response.

Here's how the Generate function works:

- The function accepts a list of prompts as its parameter.
- For each prompt, we record the start time and then use `openai.Completion.create` to generate a response from OpenAI. We limit the response to 100 tokens, but you can adjust this limit based on your needs.
- We then record the end time and calculate the time taken for the generation.
- We store the response text and the time taken in a tuple, and append this tuple to our list of responses.
- Finally, we return the list of responses.

You can call this function with a list of prompts like this:

```
prompts = ["Who won the world series in 2020?", "What's the capital of France?", "Who wrote Pride and Prejudice?"]
responses = Generate(prompts)

for i, (response, time_taken) in enumerate(responses):
    print(f"Prompt: {prompts[i]}\nResponse: {response}\nTime Taken: {time_taken} seconds\n")
```

TRY IT!

The response generated should look like the following:

```
Prompt: Who won the world series in 2020?
Response: The Los Angeles Dodgers won the 2020 World Series.
Time Taken: 0.42620301246643066 seconds

Prompt: What's the capital of France?
Response: Paris
Time Taken: 0.3195617198944092 seconds

Prompt: Who wrote Pride and Prejudice?
Response: Jane Austen
Time Taken: 0.39693474769592285 seconds
```

Click the button below when ready to submit your code.

```
import time

def Generate(prompts):
    responses = []

    for prompt in prompts:
        start_time = time.time()
        response = openai.Completion.create(
            engine="text-davinci-002", # Select the engine
            prompt=prompt,
            max_tokens=100 # Limit the response length
        )
        end_time = time.time()
        time_taken = end_time - start_time

        # Store the generated response and the time taken
        responses.append((response.choices[0].text.strip(), time_taken))

    return responses
```

Check It!Show diff

LAST RUN on 7/23/2023, 2:30:59 AM

Check 1 failed

Output:

Traceback (most recent call last):
 File ".guides/secure/GradedEX.py", line 31
 test_generate()
 File ".guides/secure/GradedEX.py", line 14
 responses = exerc.Generate(prompts)
 File "/home/codio/workspace/exerc.py", lin
 response=openai.Completion.create(
 File "/home/codio/anaconda3/lib/python3.8/
Expected:
All tests passed!

Mark as CompletedBack to dashboard

62% (19/6)

https://codio.com/sandipan-dey/monitoring-and-analyzing-chatgpt-api-usage-and-performance/tree/exerc.py

1/1